

Project Initialization & UI Design

Student Notes Manager - from database plan to visible web pages



Today's classroom goal

Create the first visible version of the Student Notes Manager and understand how Flask finds templates and static files.

Trainer focus

Students should explain the flow: browser -> Flask route -> render_template -> base template -> child page -> CSS.

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Today's learning map

Move from project idea to a clean, reusable Flask UI foundation.



Project setup

Create predictable folders and starter files.

Flask skeleton

Start the app with clear routes and config.

Templates

Use `render_template` and Jinja inheritance.

Static assets

Organize CSS, images and JavaScript.

UI pages

Build home, register and login placeholders.

Git evidence

Commit a stable Day-7 milestone.

Open problem

Create structure

Write app.py

Create templates

Run and test

Commit

Exit outcome

By the end, students must open the app in the browser and explain which file produced each visible part of the page.

Quick recall from Day 6

Database design gives the app a data blueprint. Day 7 gives it a code and UI foundation.



Day 6 deliverables

Requirements, user flow, project workflow, entities, users table, notes table, file_data table, primary and foreign keys.

Bridge question

If the database is ready, what should a user see first when the application opens?

Day 7 answer

A clean UI shell: navigation, home page, reusable layout, CSS and placeholder pages for upcoming features.

Database design

Project skeleton

Home page

Register page

Login page

Next: forms

Opening problem: why structure matters

A Flask app can run from one file, but a real project becomes messy without structure.



```
1 project/  
2   app.py  
3   home.html  
4   style.css  
5   db.py  
6   login.html  
7   random_test.py  
8   old_app.py
```

What goes wrong?

Files are hard to find.
Templates break.
CSS paths fail.
Secrets may be committed.
New team members get confused.

Better goal

Use predictable folders so Flask, trainer and students know where everything belongs.

Predictable

Reusable

Debuggable

Git friendly

Project ready

Target folder structure for Day 7

This structure supports today's UI and future authentication, notes and file features.



```
1 student_notes_manager/  
2   app.py  
3   config.py  
4   requirements.txt  
5   .env  
6   .gitignore  
7   templates/  
8       base.html  
9       home.html  
10      register.html  
11      login.html  
12  static/  
13      css/  
14          app.css  
15      js/  
16          app.js  
17      images/
```

templates/

HTML files rendered by Flask using `render_template()`.

static/

CSS, JavaScript and images served as static files.

config.py + .env

Configuration values stay separate from route logic.
Secrets should not go to GitHub.

.gitignore

Prevents `venv`, `.env`, `cache` and generated files from being committed.

Trainer checkpoint

Ask: Which folder will Flask search for `home.html`? Which folder holds `app.css`?

File responsibilities

Students should know why each starter file exists before writing code.



app.py

Creates Flask app, defines routes, starts request handling.

config.py

Keeps configuration separate from business logic.

templates/base.html

Common layout shared by all pages.

templates/home.html

Home page content inserted into base layout.

static/css/app.css

Project-specific visual styling.

requirements.txt

List of packages needed to run the project.

Starter Flask application

Begin with the smallest working app, then add UI structure around it.



New today

`render_template()` connects a Flask route to an HTML template.

Route style

`@app.get()` is clear when a route only displays a page.

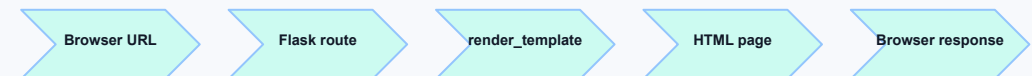
Debug reminder

`debug=True` is useful in class, but not for production.

Test URLs

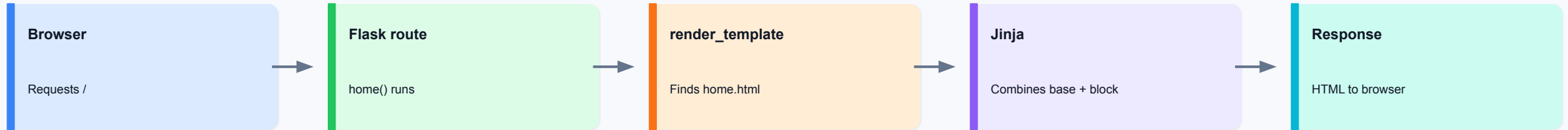
`/`
`/register`
`/login`

```
1 from flask import Flask, render_template
2
3 app = Flask(__name__)
4
5 @app.get('/')
6 def home():
7     return render_template('home.html',
8                             app_name='Student Notes
Manager')
9
10 @app.get('/register')
11 def register():
12     return render_template('register.html')
13
14 @app.get('/login')
15 def login():
16     return render_template('login.html')
17
18 if __name__ == '__main__':
19     app.run(debug=True)
```



How render_template works

This is the key request flow students must explain.



```
1 @app.get('/')
2 def home():
3     return render_template('home.html')
```

Important convention

Flask looks for templates inside a folder named templates by default.

Common mistake

Putting home.html beside app.py instead of inside templates/.

Jinja template inheritance

Use one base layout so every page shares the same header, navigation and style links.



`{{ ... }}`

Prints a value into HTML.

`{% block %}`

Creates a placeholder that child templates fill.

`url_for static`

Builds a safe path to CSS, JS or image files.

Reuse benefit

Change navbar once in base.html; all pages update.

```
1 <!-- templates/base.html -->
2 <!doctype html>
3 <html lang="en">
4 <head>
5   <title>{{ app_name or 'Student Notes Manager'
6 }}</title>
7   <link rel="stylesheet"
8     href="{{ url_for('static',
9 filename='css/app.css') }}">
10 </head>
11 <body>
12   <nav>Student Notes Manager</nav>
13   <main>
14     {% block content %}{% endblock %}
15   </main>
16 </body>
17 </html>
```



Child template: home.html

A child template extends base.html and fills only the page-specific content.



extends

Connects this page to the shared base layout.

block content

Only this part changes from page to page.

UI purpose

Students see the product before authentication is built.

Live question

Which page owns navbar: base.html or home.html?

```
1 <!-- templates/home.html -->
2 {% extends 'base.html' %}
3
4 {% block content %}
5 <section class="hero">
6   <h1>Organize your notes with Flask</h1>
7   <p>Create, search and manage learning notes
safely.</p>
8
9   <a class="btn primary" href="/register">Get
started</a>
10  <a class="btn" href="/login">Login</a>
11 </section>
12 {% endblock %}
```

Static files: CSS, JS and images

Static files do not execute Python; they are served to the browser as assets.



Folder path

static/css/app.css

Template link

`url_for("static", filename="css/app.css")`

Browser job

Downloads CSS and applies styling after HTML arrives.

Common mistake

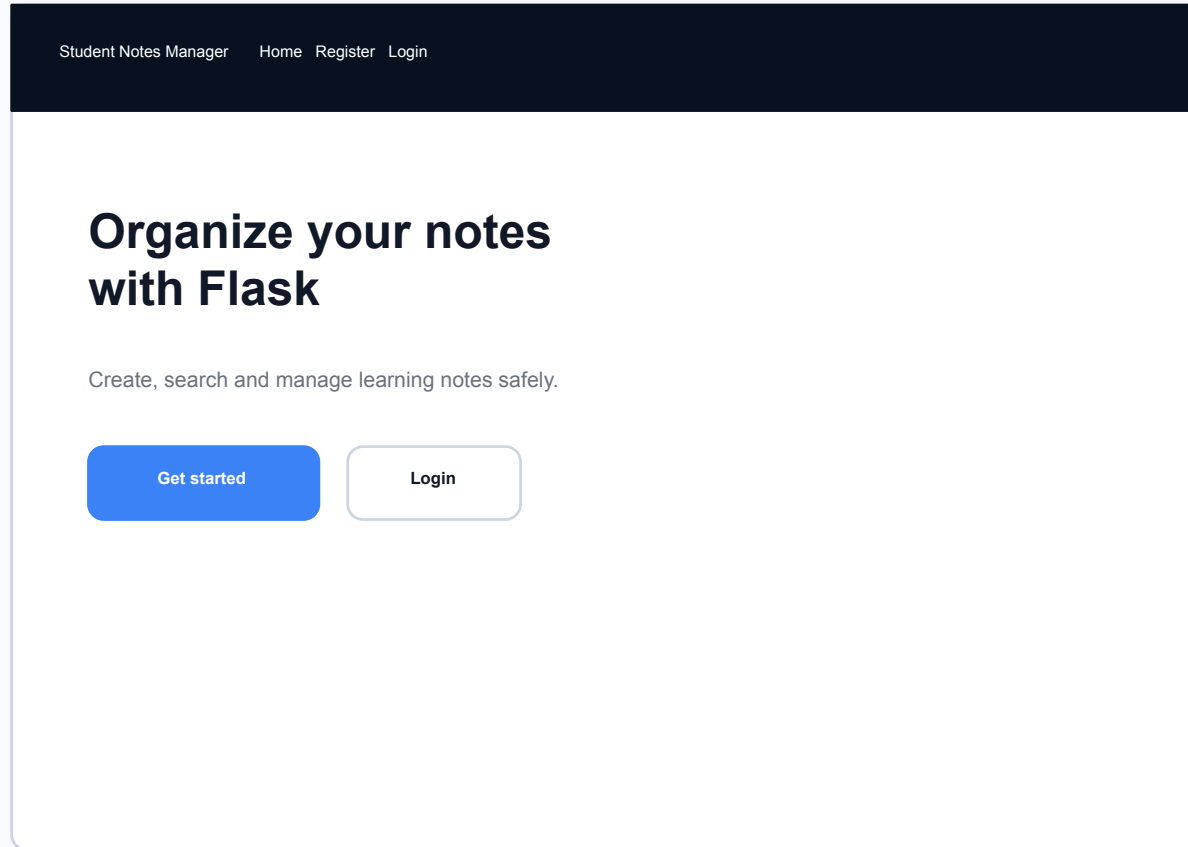
Wrong path: `/css/app.css` instead of using `url_for()`.



```
1 /* static/css/app.css */
2 body {
3     margin: 0;
4     font-family: Arial, sans-serif;
5     background: #f7fafc;
6     color: #111827;
7 }
8
9 nav {
10     padding: 16px 32px;
11     background: #111827;
12     color: white;
13 }
14
15 .hero {
16     max-width: 900px;
17     margin: 64px auto;
18     padding: 32px;
19 }
```

UI wireframe before coding

Draw the page before writing HTML. It reduces random design decisions.



Whiteboard steps

1. Draw navbar
2. Draw hero section
3. Mark buttons
4. Decide reusable elements

Teaching point

UI is not decoration only. It guides the user toward the next action.

Student question

Which sections should be common across all pages?

Register and login placeholders

Day 7 displays pages. Later days will add validation, hashing, sessions and database writes.

```
1 <!-- templates/register.html -->
2 {% extends 'base.html' %}
3 {% block content %}
4 <h1>Create account</h1>
5 <form>
6   <input placeholder="Full name">
7   <input placeholder="Email">
8   <input placeholder="Password">
9   <button>Create account</button>
10 </form>
11 {% endblock %}
```

```
1 <!-- templates/login.html -->
2 {% extends 'base.html' %}
3 {% block content %}
4 <h1>Login</h1>
5 <form>
6   <input placeholder="Email">
7   <input placeholder="Password">
8   <button>Login</button>
9 </form>
10 {% endblock %}
```

Important warning

Navigation links and route matching

Every link should point to a route that actually exists.



```
1 <nav class="navbar">
2   <a href="{{ url_for('home') }}">Home</a>
3   <a href="{{ url_for('register')
4     }}">Register</a>
5   <a href="{{ url_for('login') }}">Login</a>
6 </nav>
```

`url_for("home")`

Uses the function name, not the raw URL string.

Why safer?

If route path changes, URL generation can still be updated centrally.



Live error demo

Rename a view function or route, then show what breaks and how to read the error.

Configuration and secrets

Students should separate configuration early, even before database connection code.



```
1 # config.py
2 import os
3
4 class Config:
5     SECRET_KEY = os.getenv('SECRET_KEY',
'dev-key')
6     DB_HOST = os.getenv('DB_HOST', 'localhost')
7     DB_NAME = os.getenv('DB_NAME',
'student_notes_manager')
8
9 # app.py
10 from config import Config
11 app.config.from_object(Config)
```

```
1 # .gitignore
2 venv/
3 __pycache__/
4 .env
5 *.pyc
6 instance/
```

Rule

Never commit .env or passwords to GitHub.

Day 7 standard

Code should run locally, but secrets and environment-specific settings should stay outside the repository.

Run and test the Day 7 project

Testing today is simple but must be intentional.



```
1 # terminal
2 python -m venv venv
3 venv\Scripts\activate      # Windows
4 pip install flask
5 pip freeze > requirements.txt
6 python app.py
```

Test 1

Open <http://127.0.0.1:5000/> and verify home page.

Test 2

Click Register and Login links.

Test 3

Break a template name intentionally and read the error.

Test 4

Refresh after CSS changes and confirm style is applied.

run server

open browser

click routes

inspect errors

fix

commit

Live demo script for trainer

Use this sequence to keep the demo predictable and easy to follow.



1

Create project folder

2

Create venv and install Flask

3

Write app.py

4

Create templates/base.html

5

Create home/register/login pages

6

Add CSS and test links

Trainer narration pattern

Before typing each file, ask: Who uses this file? Browser, Flask, template engine or developer?

Common errors to catch quickly

Treat errors as teaching moments. Students must learn how to diagnose them.



TemplateNotFound

File not inside templates/ or name mismatch.

CSS not loading

Wrong static path or browser cache.

404 Not Found

URL does not match a defined route.

ImportError

Flask not installed in active virtual environment.

IndentationError

Python block structure is broken.

Secrets committed

Forgot .gitignore or committed .env.

Classroom practice plan

Students build the same structure independently, then add one small design decision.



Round 1 - reproduce

Create app.py, templates and static folder exactly as shown.

Round 2 - extend

Add an About page using the same base template.

Round 3 - explain

Explain which files changed when the About page was added.

Round 4 - debug

Trainer gives one broken path: wrong template name or wrong CSS filename.

Round 5 - commit

Commit the stable milestone with a meaningful message.

Round 6 - demo

One student opens the browser and narrates the full route-to-template flow.

Acceptance evidence

Screenshots of /, /register, /login, /about plus Git commit.

Mini project: Student Notes landing UI

This is a small visible product milestone before database-backed features.



Requirement 1

Home page should explain what the app does.

Requirement 2

Navbar should include Home, Register and Login.

Requirement 3

Register and login pages should use the same base layout.

Requirement 4

CSS should be linked through url_for static.

Requirement 5

Project should include requirements.txt and .gitignore.

Requirement 6

README should explain how to run the app.



Day 7 quick quiz

Use this as oral check or exit ticket.



1. Why do we create a templates folder?
2. What does `render_template()` return?
3. What is the role of `base.html`?
4. Why should CSS go inside `static/css/`?
5. What does `{% block content %}` mean?
6. Why should `.env` be inside `.gitignore`?
7. What is one common cause of `TemplateNotFound`?
8. Which files must change to add an About page?

```
1 from flask import render_template
2
3 @app.get('/')
4 def home():
5     return
   render_template('home.html')
```

Expected answer style

Students must answer with file names and request-flow language, not one-word answers.

Trainer check

Ask one student to draw the flow without looking at notes.

Homework assignment

Submit a working Day-7 milestone before the next class.



Task 1 - project setup

Create `student_notes_manager` with `app.py`, `templates`, `static`, `requirements.txt` and `.gitignore`.

Task 2 - pages

Implement Home, Register, Login and About routes.

Task 3 - UI

Use a shared base template and a custom `app.css` file.

Task 4 - README

Add setup commands and list all routes.

Task 5 - screenshots

Capture browser screenshots for every route.

Task 6 - Git

Commit with: Day 7: initialize project UI.

Submission evidence

GitHub link + screenshots + README.md update.

Day 7 recap

Students should leave with a running UI shell and a clear mental model.



Today we converted the Day-6 project plan into a working Flask project foundation:

1

Created predictable project structure

2

Built app.py with starter routes

3

Rendered templates with render_template

4

Used base.html and child templates

5

Served CSS through static files

6

Tested routes and committed evidence

Next class

Jinja Templates, Forms and first interactive screens.